Tipos de Datos que se utilizan en archivos Json

Aunque JSON es muy flexible, su estructura es estricta respecto a los tipos de datos que admite.

Los **6 tipos de datos** permitidos oficialmente:

---

1. Strings (Cadenas de texto)

Deben ir siempre entre **comillas dobles** (" "). Admiten caracteres Unicode y secuencias de escape (como \n para salto de línea).

- **Ejemplo:** "nombre": "Juan Pérez"

2. Numbers (Números)

JSON no distingue entre enteros y decimales (flotantes); todos se tratan como números. No se usan comillas.

- **Ejemplo:** "edad": 30 o "precio": 19.99

3. Booleans (Booleanos)

Representan valores lógicos. Solo pueden ser true o false (en minúsculas).

- **Ejemplo:** "esta_activo": true

4. Null (Nulo)

Se utiliza para indicar que un valor está vacío o no existe.

- **Ejemplo:** "segundo_apellido": null

5. Objects (Objetos)

Es una colección de pares **clave:valor** encerrada entre llaves { }. Las claves siempre deben ser strings. Los objetos pueden estar anidados.

- **Ejemplo:**

6. Arrays (Arreglos o Listas)

Es una lista ordenada de valores encerrada entre corchetes [ ]. Puede contener cualquier otro tipo de dato, incluso otros objetos o arreglos.

- **Ejemplo:** "tecnologias": ["AWS", "JSON", "Python"]

Aclaraciones

El Shell de MongoDB (MongoSH)

Cuando tú escribes comandos en la terminal de MongoDB o en un script de Node.js/Python para insertar datos:

- **Sí puedes usar comillas simples** (' ') para definir strings o claves.
- El intérprete de MongoDB es lo suficientemente flexible para aceptar ambas y convertirlas al formato binario correcto.

Ejemplos:

1. El Objeto Simple (Perfil de Usuario)

Este ejemplo muestra los tipos de datos básicos: **strings** (texto), **numbers** (números) y **booleans** (verdadero/falso).

JSON

```
{
  "id": 101,
  "nombre": "Sofía García",
  "nombre_usuario": "sofi_dev",
  "edad": 28,
  "es_premium": true,
  "correo": null
}
```

2. Estructuras Anidadas (Información de Producto)

Aquí un valor puede ser otro objeto completo. Esto es perfecto para enseñar la jerarquía de datos.

JSON

```
{
  "sku": "LAP-001",
  "producto": "Laptop ZenBook",
  "especificaciones": {
    "procesador": "Intel i7",
    "ram": "16GB",
    "almacenamiento": {
      "tipo": "SSD",
      "capacidad": "512GB"
    }
  },
  "precio": 1200.50
}
```

3. Arreglos de Objetos (Catálogo de Películas)

Este formato se utiliza bastante. Enseña cómo manejar una lista donde cada elemento es un objeto con la misma estructura.

JSON

```json
{
  "categoria": "Ciencia Ficción",
  "total_resultados": 2,
  "peliculas": [
    {
      "titulo": "Inception",
      "director": "Christopher Nolan",
      "año": 2010,
      "generos": ["Acción", "Aventura", "Sci-Fi"]
    },
    {
      "titulo": "The Matrix",
      "director": "Lana y Lilly Wachowski",
      "año": 1999,
      "generos": ["Acción", "Sci-Fi"]
    }
  ]
}
```

Puntos clave para enseñar:

- **Llaves { }:** Definen un objeto (par clave-valor).
- **Corchetes [ ]:** Definen un arreglo (lista de elementos).
- **Comas ,:** Separan elementos, pero ¡ojo!, el último elemento de un objeto o arreglo **no** debe llevar coma (un error muy común al empezar).
- **Comillas "":** En JSON, tanto las claves como los textos **deben** ir entre comillas dobles.

Resumen Visual de Tipos

Tipo	Ejemplo	Nota Importante
String	`"Hola"`	Siempre usa comillas dobles `""`.
Number	`42`	No uses comillas o será un String.
Boolean	`true`	Siempre en minúsculas.
Null	`null`	Representa ausencia de valor.
Object	`{...}`	Contenedor de pares clave:valor.
Array	`[...]`	Lista ordenada de elementos.

Dato curioso: Aunque solemos enviar fechas en JSON, el formato **no tiene un tipo "Date"**. Las fechas se envían normalmente como Strings (ej. "2026-03-22") o como Numbers (ej. un timestamp).

Un último detalle de "Pro": Las Fechas

Como mencionamos antes, JSON no tiene un tipo de dato "Fecha". ¿Cómo crees que sería mejor guardar la fecha de nacimiento de Carlos en ese JSON para que cualquier computadora del mundo la entienda sin errores?

- **A)** `"fecha_nacimiento": "22 de Marzo de 1998"`
- **B)** `"fecha_nacimiento": "1998-03-22"`
- **C)** `"fecha_nacimiento": 890524800` (Segundos transcurridos desde 1970)

¿Cuál elegirías tú y por qué?

Desde el punto de vista técnico de JSON, **las tres son válidas** porque JSON las acepta como un String (A y B) o un Number (C).

Sin embargo, en el mundo real de la programación, la elección depende de **para qué** necesites los datos. Aquí te cuento el "secreto" de cada una:

1. La opción "Humana" (A)

"22 de Marzo de 1998"

- **Problema:** Es malísima para las computadoras. Si un programador en China recibe esto, tendría que traducir "Marzo" para poder procesarlo. No se puede ordenar alfabéticamente de forma cronológica.

2. La opción "Estándar" (B) - ISO 8601

"1998-03-22"

- **Ganadora:** Es la más usada en las APIs modernas. Sigue el estándar **ISO 8601** (Año-Mes-Día).
- **Ventaja:** Las computadoras la entienden fácil, se ordena perfectamente de forma alfabética y es legible para nosotros.

3. La opción "Máquina" (C) - Timestamp

890524800

- **Uso:** Representa los segundos transcurridos desde el 1 de enero de 1970 (Era Unix).
- **Ventaja:** Es ultra eficiente para cálculos matemáticos (por ejemplo, para saber cuántos días faltan para su cumpleaños restando dos números).